System Architecture Design

We will develop a system for teleop and policy inference on real world robots. Keep all code modular so that we can easily extend to other embodiments.

Instructions

You are provided an initial git repo and SSH information in `tools.md`. The git repo is called `robosys` and is located on `beta4090` device under `~/projects/robosys/`. SSH into `beta4090` to complete the system outlined in this doc. Instructions for SSH are in `tools.md`.

Read this doc and make a plan, and then complete the system. Write some simple script tests under `robosys/test/` like making the robot arm go to a pre-defined position that I can use to test later on real hardware.

Keep track of your own progress in `progress.md`, and memory in `memory.md`. DO NOT change any files locally (not on beta4090) EXCEPT `progress.md` and `memory.md`. Add to those files often so you can clear context more often to be more efficient.

Requirements

We wish to keep as modular of a system as possible. Thus, we will disentangle specific implementations from generalizable modules as much as possible. For example, we have one API interface for all robots, defined by a base class `robot.py` that implementations like `xarm.py` will inherit.

Specifics:

These are the specifics for the current project. We will expand this repo to include other hardwares in the future, so make sure everything is modular.

Hardware:

- xArm7
- realsense d455 & d435i
- xArm built-in gripper
- xArm7 Gello (leader arm)

Software:

- Make an entry script to run in `robosys/scripts/` that instantiates a specific robot hardware and teleop device to pass into control loop.
- Use Diffusion Policy's dataset format. Have recording be a flag option when running entry script.
- Make the policy client compatible with OpenPi's server and Diffusion Policy
- Target 100 Hz control frequency, 30 Hz camera frequency, and assume 10-30 Hz policy frequency
- Use a Gym env `real_env.py` for the control loop. Reference Diffusion Policy for observation synchronization etc.
- Include done signals for teleop (triggered by keypress) by having a FSM that lets robot go from starting location -> run policy / teleop -> when teleoperator or policy marks "done" flag, reset to

starting location.
- Reward signals will be labeled by teleoperator with keypress for 0 or 1 when prompted after each episode if enabled.
- Need to have native support for interrupting the policy with teleop commands via a deadman switch. When enabled, if the deadman switch button is held, teleop will override any policy. Once released, return to policy. Default policy is holding current pose, so not pressing the deadman switch will do nothing by default. Record overridden actions and flag it as intervention.
- Keep calibration configs and other configs under `robosys/configs`

Components:

- Real robot hardware:
  - Arms
  - Grippers
  - Cameras
- Teleop:
  - Teleop device
- Data collection:
  - Buffer -> Dataset
- Policy inference:
  - Inference client
- Control loop:
  - Coordinates all of the above

Generalizability:

- unified APIs for hardware, teleop, and policy clients via inheritence (either `Protocol` or `ABC` etc.)
- for all robot interfaces, we will suport both joint space AND task space control
- we will assume robots come with Python APIs that abstract away actuator-level detail. use the APIs for corresponding robots, i.e. use `xarm-python-sdk` for xArms and `ur-rtde` for UR arms.

Project Structure

Our real-world system will a file structure similar to the following, with the goal of maximizing efficiency and modularity. This structure should be put under `robosys/src/`.

> This is not a comprehensive list of files, just a structure to get started. Add dirs/files as you see fit, but keep them organized in a similar manner.

```
cameras/
| - camera.py
| - realsense.py
dataset/
| - dataset.py
env/
| - real_env.py
grippers/
| - gripper.py
| - robotiq.py
```

```
policy/
| - client.py
| - openpi_client.py
robots/
| - robot.py
| - xarm.py
teleop/
| - gello/
| - | - gello.py
| - | - calibrate_gello.py
| - common/
| - | - teleop_device.py
| - | - interface.py
utils/
| - hardware_utils/
| - | - motors/
| - | - | - dynamixel.py
| - network_utils/
| - | - shared_memory/
| - | - udp/
```

Reference Codebases

These codebases will be useful to reference.

- Diffusion Policy
  - Good systems implementation but code is less organized. Reference for policy code and systems code (e.g. shared memory). Use Diffusion Policy's dataset format.
- Gello
  - Good modularity but limited options. Also refer to for GELLO-specific code.
- OpenPi
  - Reference for policy client code. Do not serve policies in this repo.
- LeRobot
  - Widely used with lot of compatible options.